

Week 1 Assignment — Python for Data Science & Statistics

Summer of Science 2026 · CS12

Instructions

- Submit a single, clean Jupyter notebook (`.ipynb`) with your name and roll number in the filename.
- Each question should have its code cell followed by a markdown cell with your interpretation, where asked.
- You are encouraged to refer to official documentation (NumPy, Pandas, Matplotlib, SciPy) and Google — that's the point.
- Do not share code with other mentees. Discussion is fine, copy-paste is not.

Dataset provided: `health_data.csv` — a synthetic dataset of 200 individuals with health and lifestyle attributes.

Column	Description
<code>age</code>	Age in years
<code>gender</code>	Male / Female
<code>smoker</code>	Yes / No
<code>height_cm</code>	Height in cm
<code>weight_kg</code>	Weight in kg
<code>systolic_bp</code>	Systolic blood pressure (mmHg)
<code>diastolic_bp</code>	Diastolic blood pressure (mmHg)
<code>cholesterol</code>	Cholesterol level (mg/dL)
<code>glucose</code>	Blood glucose level (mg/dL)

`exercise_hours_per_week` Weekly exercise in hours

`sleep_hours_per_night` Average nightly sleep in hours

Note: The dataset contains some missing values, handle them as part of the questions below.

Q1 — NumPy: Z-scores & Normalisation

Using **only NumPy** (no sklearn, no scipy for this question):

1. Compute the **z-scores** of the `systolic_bp` column manually. Identify all individuals whose z-score is beyond ± 2 . How many such outliers exist?
2. The dataset has two columns: `exercise_hours_per_week` and `sleep_hours_per_night`. Using **NumPy broadcasting**, normalise both columns simultaneously to the range $[0, 1]$ using min-max normalisation in a single operation (not a loop). Print the first 5 rows of the result.

Hint: min-max normalisation formula — $(x - \min) / (\max - \min)$. Figure out how to apply it to a 2D NumPy array at once.

Q2 — Pandas: Missing Values & Grouped Imputation

1. Identify all columns that have missing values. Print the count and percentage of missing values for each.
2. For each numerical column with missing values, fill the missing entries with the **median of that column grouped by gender** (i.e., missing values for Males get filled with the Male median, and the same for Females). Do not use a single global median.
3. Confirm that no missing values remain after imputation.

Hint: look up `'groupby' + 'transform'` in the Pandas documentation — there's an elegant one-liner for this.

Q3 — Matplotlib: Multi-panel Figure

Create a single figure with a **2×2 grid of subplots**. Each subplot should have a proper title and axis labels.

1. **Top-left:** Histogram of `age` with 15 bins and a KDE curve overlaid on the same axes.
2. **Top-right:** Scatter plot of `weight_kg` vs `systolic_bp`, with points coloured differently for smokers and non-smokers. Include a legend.
3. **Bottom-left:** Box plot of `cholesterol` for each `gender` side by side.
4. **Bottom-right:** Bar chart showing the average `glucose` level across age groups — bin ages into decades (20s, 30s, 40s, 50s, 60s) and plot one bar per group.

Hint: use `plt.subplots(2, 2)`, look up `pd.cut` for age binning, and `matplotlib` colour arguments for scatter grouping.

Q4 — Distributions: Skewness & Transformation

1. For the `glucose` column, compute the mean, median, standard deviation, and skewness (use `scipy.stats.skew` or Pandas `.skew()`). Based on the skewness value, classify the distribution.
 2. Plot the original `glucose` distribution as a histogram with a KDE curve.
 3. Apply a **log transformation** to `glucose` (use `numpy.log`). Recompute the skewness and plot the transformed distribution the same way.
 4. In a markdown cell, explain: what did the log transformation do to the skewness, and why does this matter when building ML models?
-

Q5 — Central Limit Theorem: Simulation

This question asks you to *demonstrate* the CLT through code, not just describe it.

1. From the `systolic_bp` column (use the full column, after imputation), draw **1000 random samples** of size $n=30$ (with replacement) and compute the mean of each sample. Store these 1000 sample means.
2. Plot the distribution of sample means as a histogram.
3. Compute the mean and standard deviation of the sample means. Compare the standard deviation of sample means to `population_std /  $\sqrt{30}$` — are they close?
4. In a markdown cell, answer: even though `systolic_bp` might not be perfectly normal, why does the distribution of sample means look approximately normal? What does this tell you about the CLT?

Q6 — Hypothesis Testing: Smokers vs Non-smokers

1. State clearly: what is your **null hypothesis** and **alternative hypothesis** regarding the mean `systolic_bp` of smokers vs non-smokers?
2. Use `scipy.stats.ttest_ind` to perform an **independent samples t-test**. Report the t-statistic and p-value.
3. At a 5% significance level, do you reject or fail to reject the null hypothesis? Interpret the result in plain English — what does it mean clinically?
4. Compute a **95% confidence interval** for the mean `systolic_bp` of each group separately using `scipy.stats.t.interval`. Report and interpret both intervals.

Hint: look up `scipy.stats.ttest_ind` and `scipy.stats.t.interval` in the SciPy documentation.

Q7 — Correlation Analysis

1. Compute the **correlation matrix** of all numerical columns. Visualise it as an annotated heatmap using `seaborn.heatmap`.
 2. From the matrix, identify the **top 3 strongest correlations** (exclude self-correlations and duplicate pairs).
 3. For the strongest correlated pair, create a scatter plot and overlay a **line of best fit** using `numpy.polyfit` and `numpy.poly1d`. Display the equation of the line on the plot.
 4. In a markdown cell, answer: correlation measures linear association — identify one pair from the matrix where you think the relationship might *not* be linear even if the correlation value is moderate. How would you investigate this further?
-